

Why a Capable CPU Boots With Broken Clocks

The Legion Pro 5 TSC bug and the case for open firmware

Hasin Ishraq

Undergraduate CSE student, Islamic University of Technology (IUT), Bangladesh

hasinishraq001@gmail.com

28 September 2026

Abstract

On every boot, the firmware of a Lenovo Legion Pro 5 16ADR10 (BIOS RLCN32WW) hands Linux a processor whose cores disagree about the time by about two seconds. The system journal recorded this on all 69 unfixed boots with a complete kernel log between 28 August and 26 September 2026, across six Linux kernels. The hardware is not at fault: the same cores agree perfectly after resuming from sleep, and a 0.6-second pre-boot tool that moves one counter makes Linux accept them on every boot. The bug is a few lines of firmware logic, but the owner cannot fix it, because the CPU is fused to run only Lenovo-signed firmware. That is the argument this report makes for open firmware: source code the community can read and patch, delivered through the same signed, recoverable update path vendors already use.

1 The boot history

The systemd journal kept the kernel log of 80 boots from 28 August to 28 September 2026, and every boot without a fix failed the kernel’s TSC check. Since the pre-boot fix, `tscsync` [1], reached its current form (v1.3), all 7 boots kept the TSC. Table 1 groups the boots by what ran before Linux.

Phase	Dates (2026)	Boots	What the kernel did
No fix	28 Aug – 26 Sep	68	Rejected the TSC on all 67 boots with a full log; warp 5.04–6.50 billion cycles
<code>tscsync</code> v1.0, measure only	26 – 27 Sep	2	Rejected; the tool saw all 31 other cores 5,690,270,7xx cycles ahead of CPU 0, and the kernel measured 5,690,270,706
<code>tscsync</code> v1.1, sync	27 Sep	1	Rejected: a 25-cycle warp remained
<code>tscsync</code> v1.2, sync	27 Sep	2	1 kept the TSC; 1 rejected it (25-cycle warp)
<code>tscsync</code> v1.3, sync	27 Sep	3	Kept the TSC after a cold start, a clean shutdown and a reboot
<code>tscsync</code> v1.4, sync	27 Sep	1	Kept the TSC, also through 2 resumes from sleep
<code>tscsync</code> v1.5, sync	27 Sep	2	Kept the TSC; 556 ms per boot
<code>tscsync</code> v2.0.0, sync	28 Sep	1	Kept the TSC; all 31 cores within -15 to $+1$ cycles of CPU 0; 579 ms

Table 1: Every boot in the journal, grouped by what ran before Linux. Measure mode reads the counters and writes nothing, so those boots count as unfixed.

The one unfixed boot without a warp line has a journal that starts 6.3 seconds in, after the kernel’s check. The failures of v1.1 and v1.2 came from the tool’s precision, not the hardware; they set the 10-cycle target later versions use.

2 The fingerprint of a firmware bug

The warp is exactly proportional to how long the firmware ran, with no fixed part: 833.31 million cycles per second of firmware time, intercept -0.41 ms, $R^2 = 0.999999$, largest miss 0.18 ms (Figure 1).

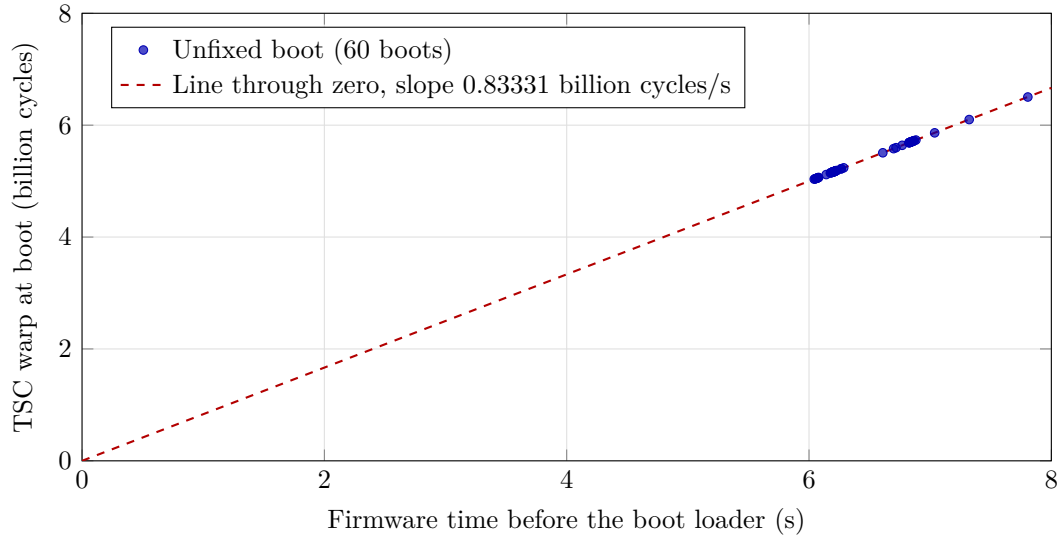


Figure 1: TSC warp the kernel measured against systemd’s firmware time, for every unfixed boot that logged both (28 August to 27 September 2026). The 9 unfixed boots without a point are PackageKit offline-update boots, which reboot before systemd records the firmware time.

The firmware time comes from the firmware’s own boot-performance table (ACPI FPDT), which it measures with CPU 0’s TSC. At a TSC rate of about 2.495 GHz, the warp is one third of CPU 0’s own count: when the boot loader starts, CPU 0’s counter reads three quarters of every other core’s. A delay would add a constant; this is a scale error in how CPU 0’s counter is set up or counted.

Three more measurements point at the same place:

- **Only CPU 0 is off.** Measured from GRUB before Linux, all 31 other cores agreed within about 40 cycles, across both chiplets, and all were 5.2–5.7 billion cycles ahead of CPU 0.
- **Nothing after the firmware changes it.** The pre-boot tool measured 5,690,270,7xx cycles; the kernel measured 5,690,270,706 moments later on the same boot.
- **The resume path is correct.** After a suspend and resume, the firmware restarts all counters and every core agrees; the kernel re-checks each core and keeps the TSC.

3 The hardware is capable

Nothing about the Ryzen 9 8945HX prevents synchronized counters. Four facts from this machine show it:

- **The counters are invariant.** The CPU reports an invariant TSC (`constant_tsc`, `nonstop_tsc`): every core’s counter runs at a fixed rate regardless of power state, so once the counters agree they stay in agreement.

- **The firmware already gets it right once.** Resuming from sleep (S3) runs a different firmware path, which starts all 32 counters together. Linux re-checks every core on resume and keeps the TSC.
- **One write fixes it.** A 0.6-second pre-boot tool moves only CPU 0's counter forward by about 5.2 billion cycles. The kernel then accepts the TSC on every boot, and user-space probes see all cores within a few dozen cycles for the rest of the session.
- **Windows hides the gap.** Windows synchronizes the counters itself at boot and exposes that as a boot option, `tscsyncpolicy` [4]. A laptop sold and tested with Windows never shows the symptom.

4 Why this BIOS fails and others don't

On AMD Zen, the firmware alone is responsible for handing over synchronized counters. This BIOS's cold-boot path leaves CPU 0 behind, and nothing downstream catches it.

Who is allowed to fix an offset. Intel CPUs have a per-core register, `IA32_TSC_ADJUST`, and Linux uses it to pull a mismatched core back into line when it comes online [2]. Zen CPUs have no such register (this machine reports `tsc_adjust=0`). Without it, Linux tests once and falls back to HPET; the kernel's timekeeping maintainer rejected writing the counters directly from the kernel as "flaky and unreliable" [3]. So on AMD, only firmware can get this right.

What this BIOS does differently. The data narrows it down, though not to a line of code: the error is a clean scale factor on CPU 0 alone, produced during the pre-boot firmware phase and absent from the resume path. The likeliest explanation is that the cold-boot path initializes or clocks CPU 0's counter differently from the other 31 cores and never re-synchronizes them before handing over. Confirming it needs Lenovo's source.

Why it ships anyway. Windows re-synchronizes the counters itself at every boot, so a laptop validated on Windows passes. Firmware that syncs correctly (the resume path here, and most other vendors' boot paths) leaves nothing for either operating system to repair, which is why most AMD laptops never show this warning.

It is not the first time. The same symptom was reported on the IdeaPad 5 2-in-1 14AHP9 (BIOS P1CN32WW) and on another Legion Pro 5 16ADR10, with a Ryzen 9 8940HX, on the same RLCN32WW BIOS [5]. Users also report that Lenovo fixed the same class of bug by BIOS update on the ThinkPad A485, E495 and T14s Gen 1 AMD models; those forum pages could not be opened for this report, so treat that history as reported, not verified.

5 What it costs users

On HPET, every clock read is a trip to a slow timer device, and the kernel and desktop read the clock tens of thousands of times a second (Table 2).

Forcing the TSC on anyway (`tsc=reliable`) is not a workaround: the counters really are two seconds apart. The IdeaPad owner who first reported this found that it causes audio and video stutter during playback and erratic kernel timers [5].

Measure on this laptop	HPET (firmware as shipped)	TSC (after the pre-boot fix)
<code>clock_gettime</code> from Python, per call	1,241 ns	144 ns
CPU temperature, mostly idle	66–72 °C (owner’s observation)	59–63 °C; 59.9 °C measured, 97.6 % of the time in the deepest idle state

Table 2: Cost of the HPET fallback. Both timing figures include Python’s own call overhead, so the raw difference is larger. The temperature comparison is an owner’s observation, not yet a controlled test; `tscsync` includes a benchmark script (`tools/idle-bench.sh`) for one.

6 Why owners cannot fix it themselves

The fix is small, but on this laptop only Lenovo can ship it: the CPU is permanently fused to Lenovo’s firmware signing key.

- **Platform Secure Boot is fused.** The CPU’s security processor reports `fused_part = 1`. Under AMD Platform Secure Boot, the security processor checks the firmware’s signature against a key hash burned into the chip before any x86 code runs, and the fuses cannot be reset [6, 7]. A patched BIOS would simply not start.
- **The flash chip is guarded.** `rom_armor_enforced = 1`: writes to the BIOS chip go through the security processor, so software cannot write a modified image. Only an external programmer clipped to the board could restore the original.
- **The code is closed.** AMD’s platform initialization code (AGESA) and Lenovo’s firmware are proprietary. AMD’s open replacement, openSIL, is a proof of concept on a server reference board today and does not cover this laptop [8].

The security model is sound in itself: it stops malware from implanting itself into firmware. The problem is that it puts every fix in one vendor’s queue, even when the owner has found the bug, measured it and demonstrated the fix.

7 The case for open firmware that updates safely

The fix for this bug is probably a few lines in the boot path: the code that syncs the TSCs after S3 resume already works. Only Lenovo can make that change today, and owners have reported it on Lenovo’s forum and by email. Meanwhile every owner of this model runs a clock that is wrong on every cold boot. Open firmware would change who can find and fix bugs like this. It does not have to make firmware less safe.

7.1 “Open” does not mean “unsigned” or “flash it yourself”

The usual objection is that open firmware means owners flashing random ROMs and bricking laptops. That mixes up two things:

- **Who can read and rebuild the source.** This is what “open” means here.
- **Who can sign a build the machine will accept.** This can stay with the vendor.

AMD Platform Secure Boot and Lenovo’s signing keys can stay exactly as they are. The vendor still builds, signs and ships every update. The difference is that the source is published and the build is reproducible. Anyone can then check that the signed binary matches the source, and anyone can find a bug, write the patch and send it to the vendor. The vendor still decides what ships. Linux distributions already work this way with signed kernels under Secure Boot.

7.2 What “doesn’t brick the device” needs, and where it already exists

Lenovo’s own BIOS updates are safe because of the delivery mechanism, not because the source is closed. Every part of that mechanism already exists in open form (Table 3).

Safety property	How it works	Open implementation in use today
Only vendor-signed images are accepted	The update is checked against a key in firmware before it is written	Chromebook verified boot checks each firmware copy’s signature with a key in the read-only boot stub [11]; Dasharo supports verified boot and signed releases [10]
A failed update cannot leave the machine unbootable	A read-only recovery region plus two copies of the writable firmware (A/B)	Chromebooks: “The writable portion of the firmware should exist in two copies. In the event the first copy is corrupt, the device can boot normally off the second copy.” [11]
Updates are applied by the firmware, not by a flashing tool in the OS	UEFI capsule update: the OS stages the image and the firmware applies it on the next boot	fwupd calls the UEFI <code>UpdateCapsule</code> service [13]; Dasharo ships capsule updates [10]
Owners get updates without Windows or vendor tools	A signed catalog and signed update files	The LVFS, used by over one hundred vendors, signs each capsule, and fwupd installs it on Linux [12, 13]
Anyone can check the binary against the source	Reproducible builds	Dasharo documents how to verify signatures and reproduce its builds [10]

Table 3: The properties that make a firmware update safe, and open implementations of each.

None of this is experimental:

- Every Chromebook ships **coreboot** [9] with an A/B firmware layout and a read-only recovery path, on millions of devices.
- **Dasharo** (3mdeb) ships coreboot with EDK II UEFI on laptops sold by NovaCustom and others, with fwupd and capsule updates [10].
- Lenovo already publishes firmware for many ThinkPad models on the **LVFS**. Legion models are generally not there, so Linux owners of this laptop cannot even install a BIOS update without Windows or a USB tool.

7.3 What blocks this on AMD laptops today

The part of the firmware that holds this bug is the CPU and platform initialization code: AMD’s AGESA, integrated by Insyde and Lenovo. That code is closed.

AMD’s open replacement, **openSIL**, is public and MIT-licensed, but its repository describes it as a proof of concept for a server reference board (Genoa), “NOT for use in production firmware” [8]. It has no client (laptop) CPU support. Press reports say client support is planned for a future CPU generation; AMD has not shipped it. Until then, even a vendor willing to open its firmware could not open the part that syncs the TSCs.

So the realistic path has two steps:

1. **Now:** vendors publish the platform code they own, ship through the LVFS, and test cold boot on Linux.

2. **When openSIL covers client CPUs:** laptops built on coreboot or EDK II with openSIL, vendor-signed, with A/B recovery, delivered as capsules. A bug like this one could then be found, fixed and reviewed in public, and still reach owners only through a signed, recoverable update.

7.4 Why it matters beyond one laptop

This bug shows what closed firmware costs:

- **The hardware is fine and the fix is known.** The same firmware already syncs the TSCs correctly on resume.
- **Windows hides the problem.** Windows resyncs the TSCs itself, so a vendor that tests only on Windows never sees it.
- **Only the vendor can fix it, and the vendor has little reason to.** Owners are left with a pre-boot workaround (tscsync) that has to be maintained, signed and installed outside the firmware.

With open source, reproducible builds and vendor signing, the person who found the bug could have sent the fix, and the vendor could have reviewed, signed and shipped it through the same safe update path it uses today.

8 Recommendations

8.1 For Lenovo and other vendors

1. **Fix the cold-boot TSC sync.** Apply the same sync between CPU 0 and the other cores that the S3 resume path already does, before handing off to the operating system. The kernel's check (`check_tsc_sync_source`) should pass on every core with no help from the OS.
2. **Test cold boot on Linux.** A single boot of a current Fedora or Ubuntu live image shows TSC warp in the kernel log. Windows hides the bug because it resyncs the TSCs itself.
3. **Ship firmware through the LVFS** for consumer lines too, not only ThinkPads, so Linux owners get fixes through fwupd.
4. **Publish the platform code you own**, with reproducible builds, and move to openSIL when AMD supports client CPUs.

8.2 For owners of affected machines

- **Check** the kernel log of the current boot:

```
journalctl -k -b | grep -iE 'tsc|clocksource'
```


Marking TSC unstable or TSC warp, followed by `Switched to clocksource hpet`, means you are affected.
- **Report it** to the vendor with the model, BIOS version and that log line. A workaround does not replace a firmware fix.
- **Work around it** with tscsync [1]: install it in measure mode first, then switch to sync mode. It writes nothing to firmware and disables itself if a boot fails.
- **After every BIOS update**, switch back to measure mode for one cold boot. If every core shows OK, the vendor has fixed the bug and tscsync can be removed.

A Methodology and data

All data and scripts are in the `tscsync` repository under `docs/report/` [1].

- **Boot history** (`data/boots.csv`, written by `extract-boots.sh`): one row per boot in the systemd journal of this machine, every boot since installation. For each boot: the kernel version, the warp from Measured N cycles TSC warp between CPUs, systemd's firmware time from Startup finished in ... (firmware) (which systemd reads from the ACPI FPDT), the `tscsync` version and mode, and whether the kernel kept the TSC.
- **Warp against firmware time** (`analyze.py`): an ordinary least-squares fit of the warp against the firmware time over every unfixed boot that logged both (60 boots). It writes the plotted points to `data/warp-vs-firmware.csv`.
- **Per-core offsets**: measured before Linux starts by `tscsync` in measure mode, and after boot by `tools/tscprobe.c`. Both use a paired-read test between CPU 0 and each other core, giving the offset in cycles and its direction.
- **Resume path**: the kernel's TSC check on S3 resume, with and without the workaround.
- **Limits**: one machine and one BIOS version (RLCN32WW). The other models are listed from owner reports and were not measured here.

References

- [1] H. Ishraq, *tscsync: fix TSC warp between CPUs before Linux boots*, <https://github.com/Zanasin/tscsync> (see `docs/DESIGN.md` and `docs/HARDWARE.md`).
- [2] Linux kernel, `arch/x86/kernel/tsc_sync.c`, https://github.com/torvalds/linux/blob/master/arch/x86/kernel/tsc_sync.c.
- [3] Linux kernel mailing list, discussion of TSC synchronization on AMD CPUs without TSC_ADJUST, August 2022, <https://lkml.iu.edu/hypermail/linux/kernel/2208.3/00630.html>.
- [4] Microsoft, *BCDEdit /set (tscsyncpolicy)*, <https://learn.microsoft.com/en-us/windows-hardware/drivers/devtest/bcdedit--set>.
- [5] Lenovo Community, *1 second TSC warp on IdeaPad 5 2-in-1 14AHP9 due to bad initialization in BIOS forces HPET fallback*, <https://forums.lenovo.com/t5/Other-Linux-Discussions/1-second-TSC-warp-on-IdeaPad-5-2-in-1-14AHP9-due-to-bad-initialization-in-BIOS-forces-HPET-fallback/m-p/10021248>.
- [6] IOActive, *Exploring AMD Platform Secure Boot*, <https://www.ioactive.com/exploring-amd-platform-secure-boot/>.
- [7] ServeTheHome, *AMD PSB vendor locks EPYC CPUs for enhanced security at a cost*, <https://www.servethehome.com/amd-psb-vendor-locks-epyc-cpus-for-enhanced-security-at-a-cost/>.
- [8] AMD, *openSIL*, <https://github.com/openSIL/openSIL>.
- [9] coreboot documentation, <https://doc.coreboot.org/>.
- [10] 3mdeb, *Dasharo documentation*, <https://docs.dasharo.com/>.
- [11] The Chromium Projects, *Firmware boot and recovery*, [https://www.chromium.org/chromiumos-design-docs/firmware-boot-and-recovery/](https://www.chromium.org/chromium-os/chromiumos-design-docs/firmware-boot-and-recovery/).
- [12] fwupd, <https://fwupd.org/>.
- [13] Linux Vendor Firmware Service, *Introduction*, <https://lvfs.readthedocs.io/en/latest/intro.html>.